

Session 6

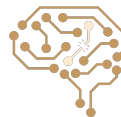
Local Search



CE 417-Artificial Intelligence
Sharif Univ. of Tech.

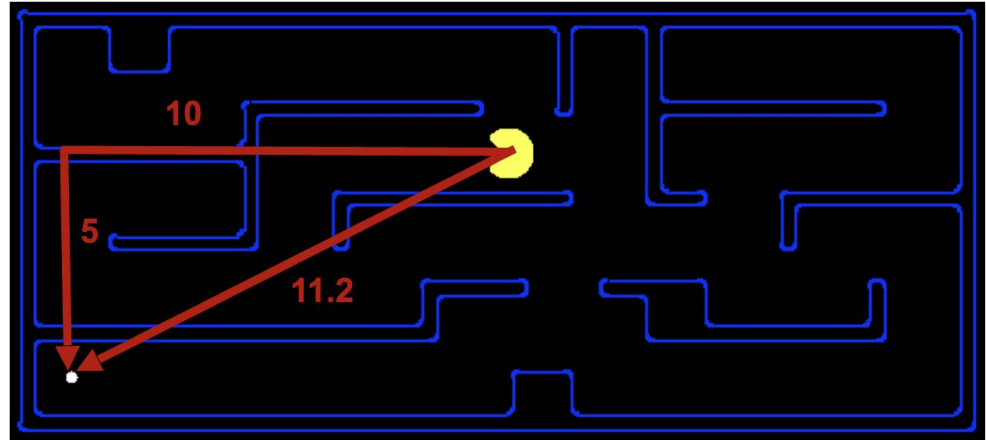
Arash Marioriyad
30 Farvadin 1405

Some of Slides have been adopted from Washington U.
CSE-573 and Sharif U. CE417



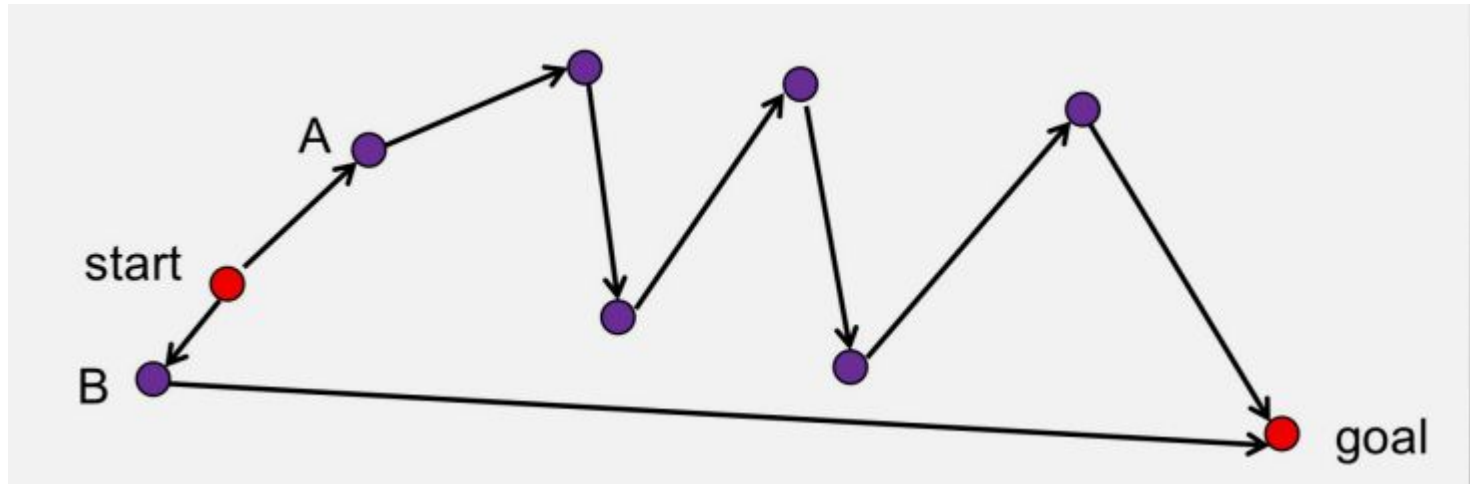
Search Heuristics

- A function $h(s)$: **estimates** how close a state is to a goal
- For each search problem we design/consider particular heuristics
- Example:
 - Manhattan distance
 - Euclidean distance



Greedy Algorithm

- Only Uses Heuristic Information
- Expand the node that seems closest
- Not Optimal



A* Search



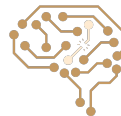
UCS



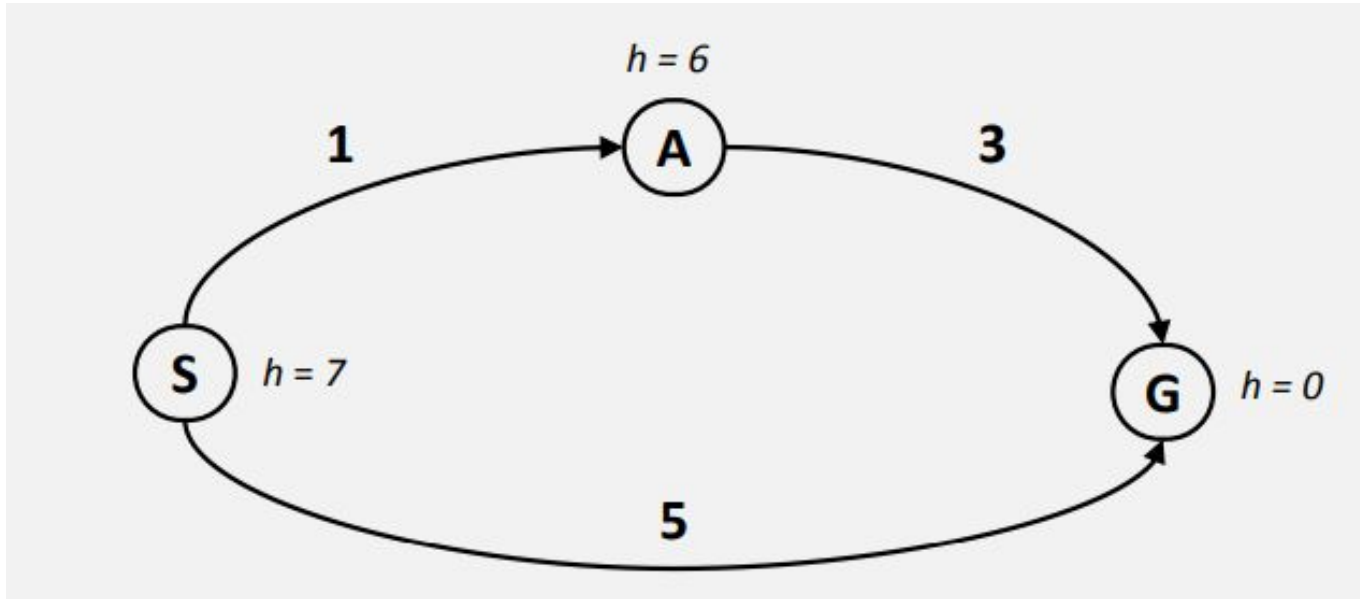
Greedy



A*



Is A* Optimal?

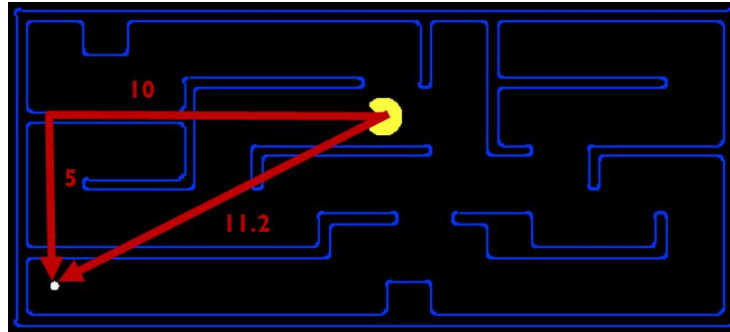


Admissible Heuristics

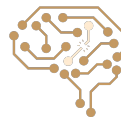
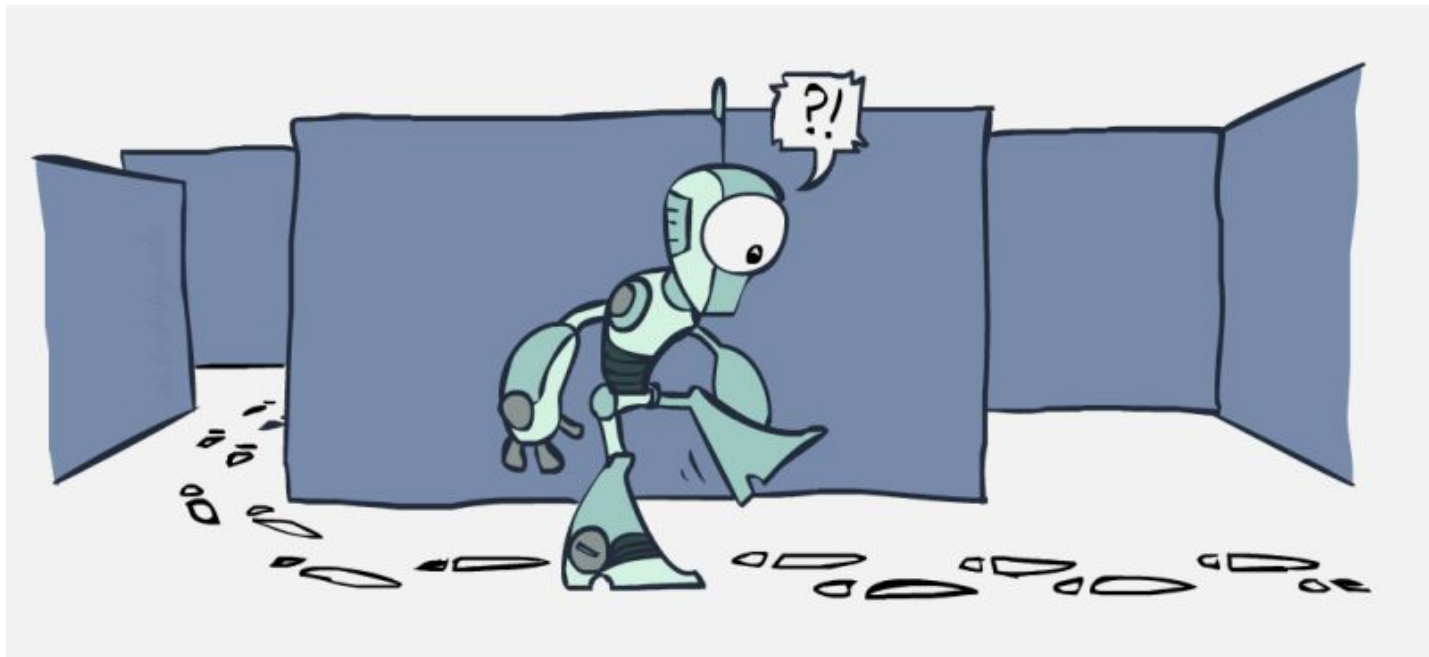
- A heuristic h is admissible (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

- where $h^*(n)$ is the true cost to a nearest goal
- Examples:

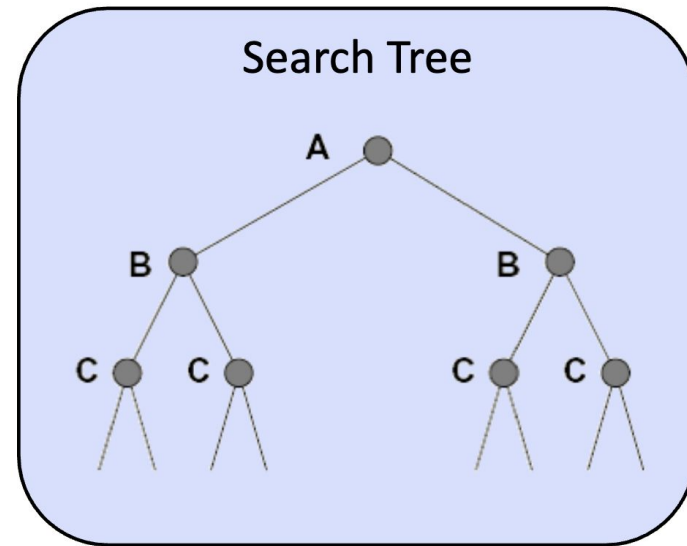
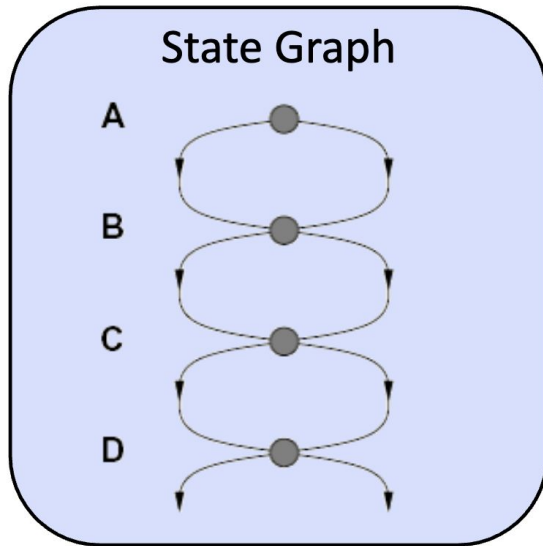


Graph Search



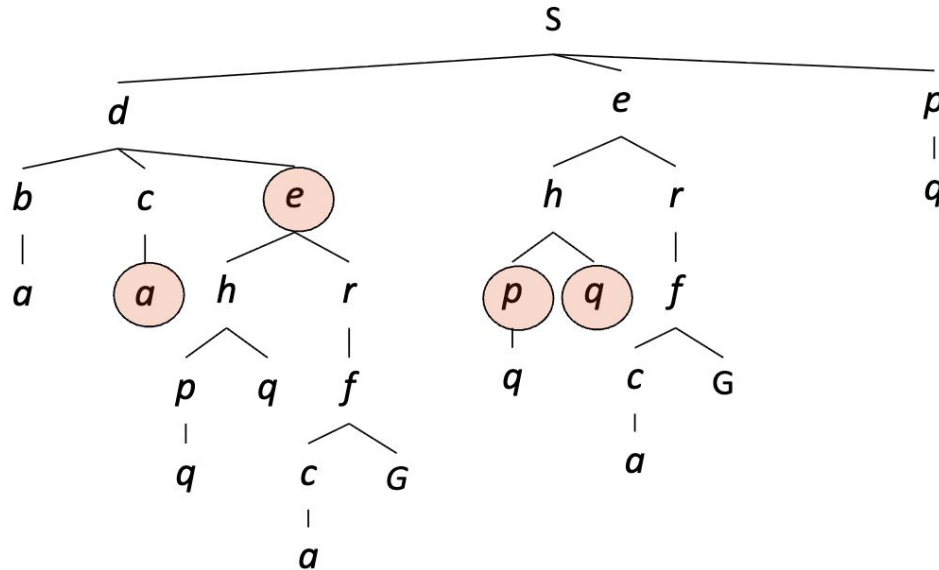
Problem of Tree Search

- Failure to detect repeated states can cause exponentially more work.



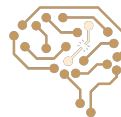
Solution: Graph Search

- Idea: Never expand a same state twice



Graph Search Algorithm

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
    end
  end
end
```



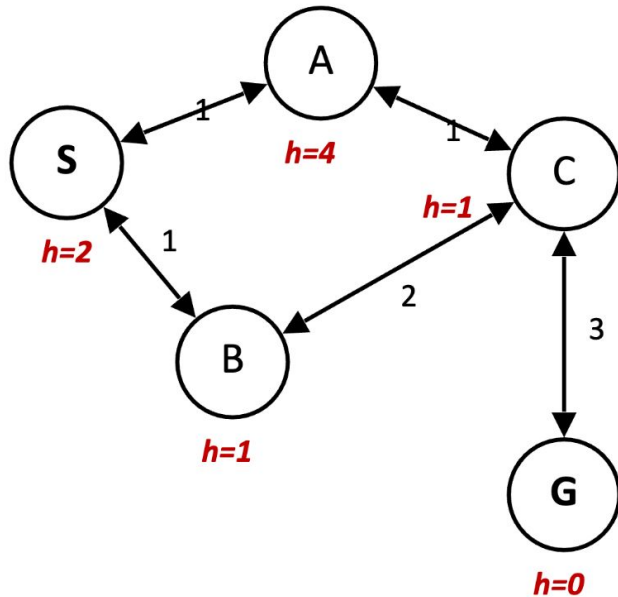
Graph Search Implementation

- Tree search + set of previously expanded states (“closed set”)
- Expand the search tree node-by-node, but...
- Before expanding a node, check to make sure its state has never been expanded before
- If not new, skip it, if new add to closed set

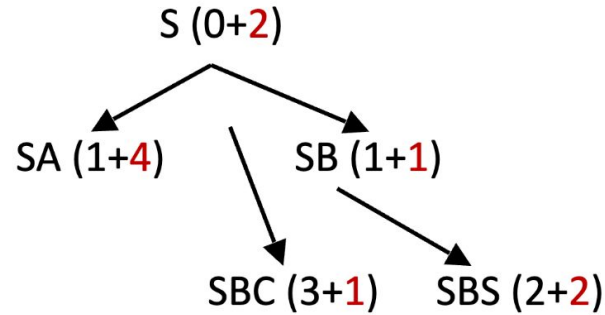


A* Graph Search Gone Wrong?

State space graph



Search tree

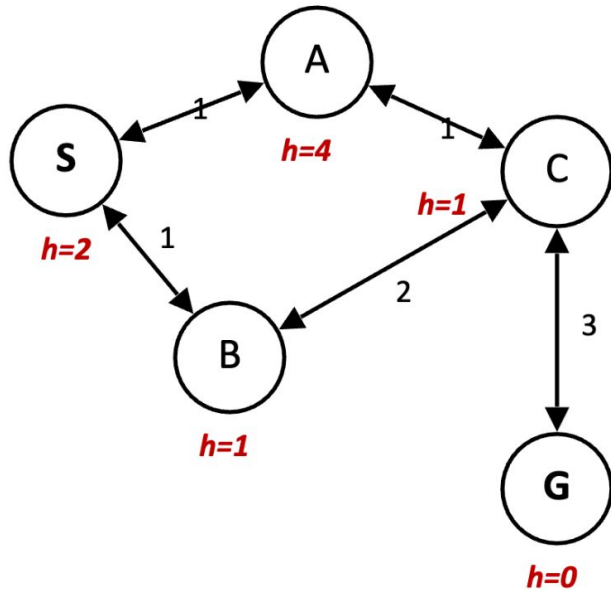


Closed set
{ S B }

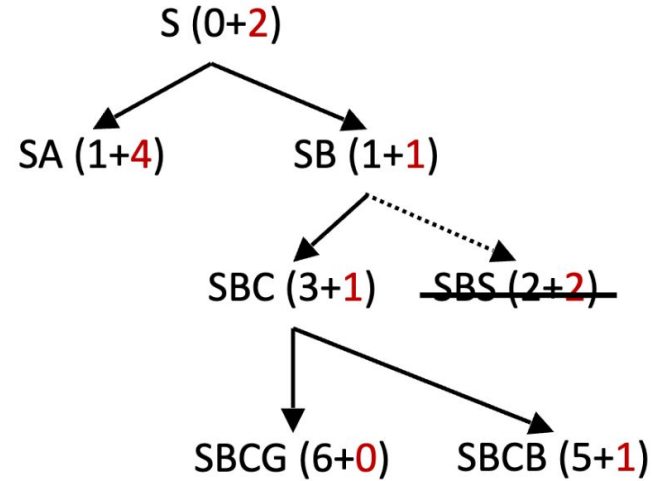


A* Graph Search Gone Wrong?

State space graph



Search tree

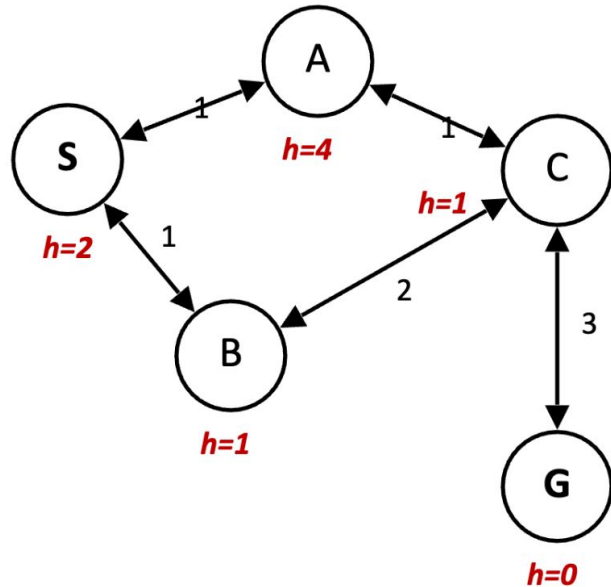


Closed set
{ S B C }

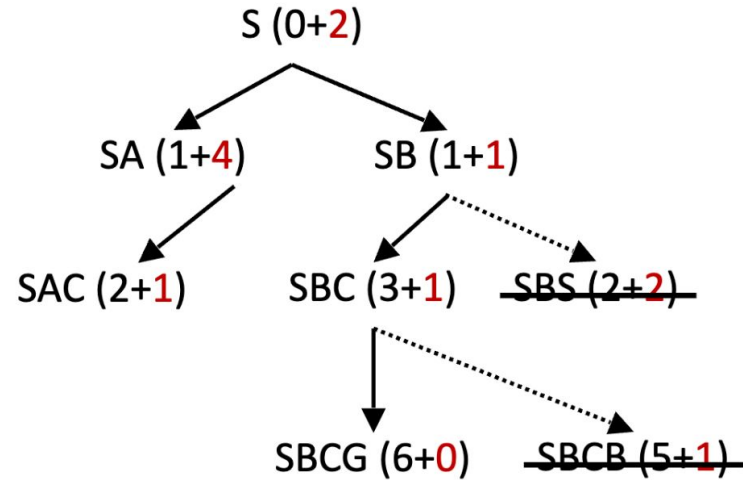


A* Graph Search Gone Wrong?

State space graph



Search tree

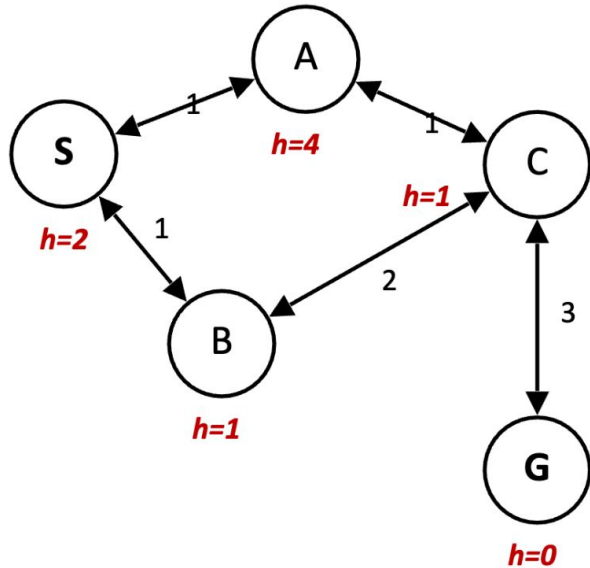


Closed set
{ S B C A }

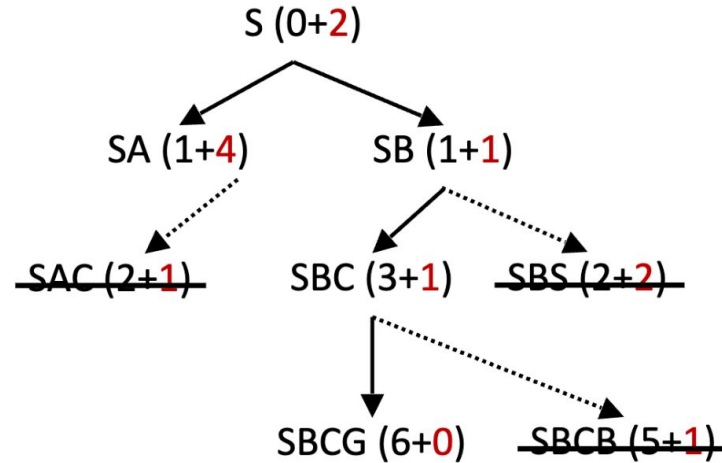


A* Graph Search Gone Wrong?

State space graph



Search tree

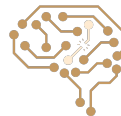
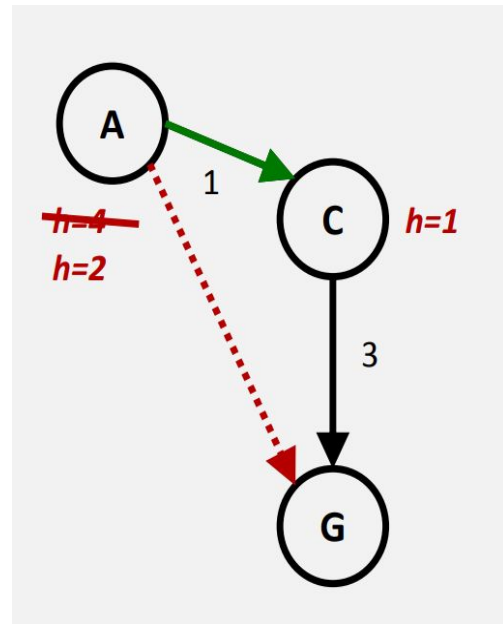


Closed set
{ S B C A }



Consistency (Monotonicity) of Heuristic

- Main idea: estimated heuristic costs $\leq$ actual costs
 - **Admissibility:** heuristic cost $\leq$ actual cost to goal
 - $h(A) \leq \text{actual cost from A to G}$
 - **Consistency:** heuristic “arc” cost $\leq$ actual cost for each arc
 - $h(A) - h(C) \leq \text{cost}(A \text{ to } C)$
- Consequences of consistency:
 - The f value along a path never decreases
 - $h(A) \leq \text{cost}(A \text{ to } C) + h(C)$
 - A* graph search is optimal. Why?



Optimality of A* Graph Search

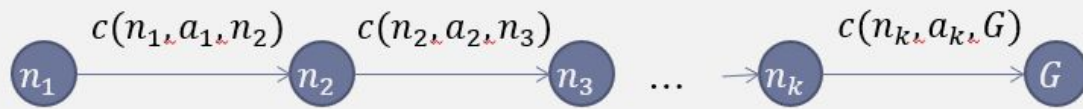
- If $h(n)$ is monotonic, then the values of f along any path are non-decreasing.
- A* expands all nodes in order of non-decreasing f value.
- Optimal goal, G , has the lowest $f(G)$ among all the goals, when selected for expansion.
- Whenever A* selects node n for expansion, the optimal path to that node has been found.

=> Optimal goal G will be expanded first among all the goals.

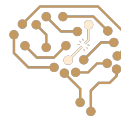


Consistency Implies Admissibility

- All consistent heuristic functions are admissible
- Nonetheless, most admissible heuristics are also consistent



$$\begin{aligned} h(n_1) &\leq c(n_1, a_1, n_2) + h(n_2) \\ &\leq c(n_1, a_1, n_2) + c(n_2, a_2, n_3) + h(n_3) \\ &\quad \dots \\ &\leq \sum_{i=1}^k c(n_i, a_i, n_{i+1}) + h(G) \Rightarrow h(n_1) \leq \text{cost of (every) path from } n_1 \text{ to goal} \\ &\leq \text{cost of optimal path from } n_1 \text{ to goal} \end{aligned}$$



Example: The 8-puzzle

- $h_1(n)$ = Number of Misplaced Tiles $\rightarrow$ Consistent?
- $h_2(n)$ = Total Manhattan Distance $\rightarrow$ Consistent?

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State



Optimality

- Tree search:
 - A* is optimal if heuristic is admissible
 - UCS is a special case ($h = 0$)
- Graph search:
 - A* optimal if heuristic is consistent
 - UCS optimal ($h = 0$ is consistent)
- Consistency implies admissibility

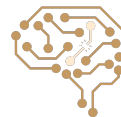
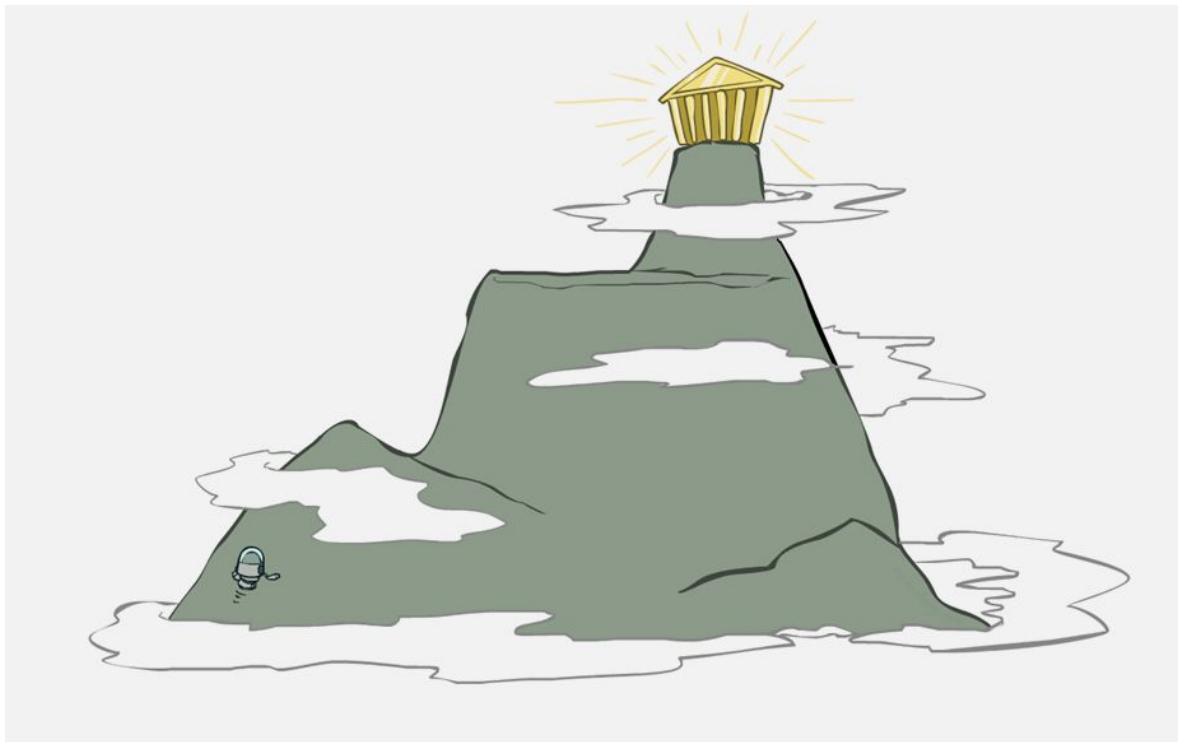


A* Summary

- A* uses both backward costs and (estimates of) forward costs
- A* is optimal with:
 - Admissible heuristics for tree search
 - Consistent heuristics for tree and graph searches
- Heuristic design is key: often use relaxed problems
- Among all search algorithms that have access to the heuristic function h , A* expands fewest possible nodes



Local Search



Two Viewpoints of States

- **Black box** of information about the agent and world
- **White Box** of Information about the agent and world
 - There are variables
 - There are values (domain for each variable)
 - States are different assignment of values to variables
 - Each assignment of a variable, is an action
 - The goal state(s) are specific forms of assignments

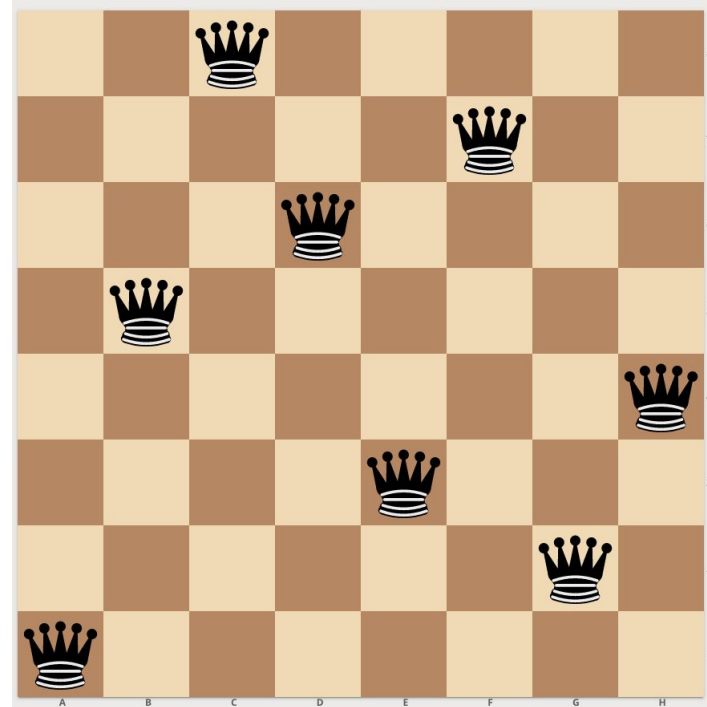
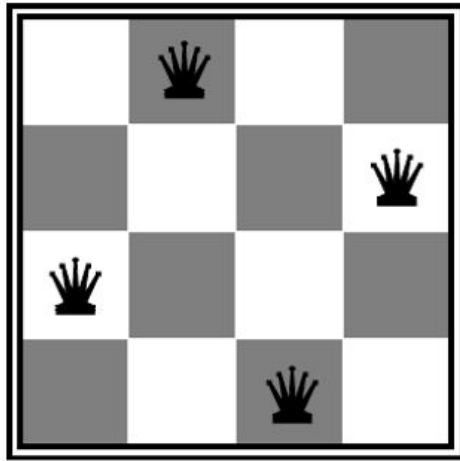


Partial States vs Complete States

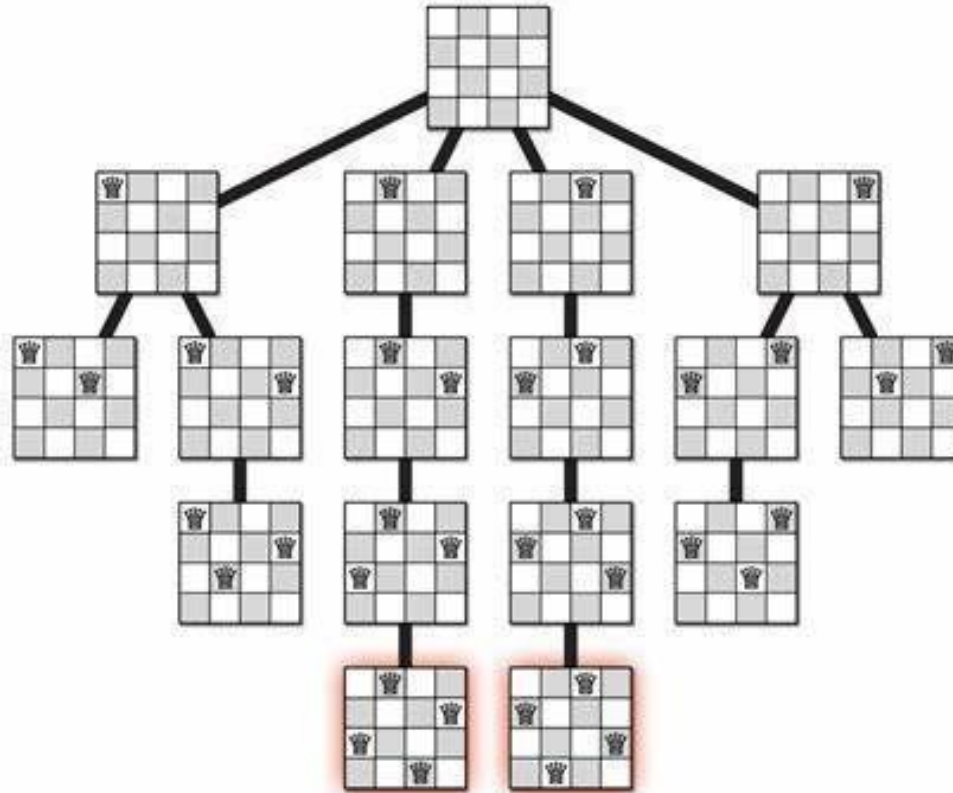
- **Partial State:** There are at least one unassigned variable
- **Complete state:** All variables have values
- Start state, is the state where no variable has value
- Goal states are a subset of complete states



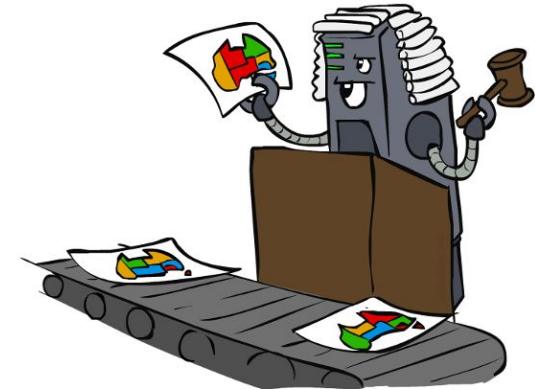
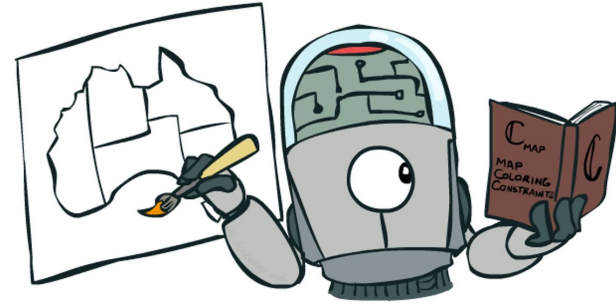
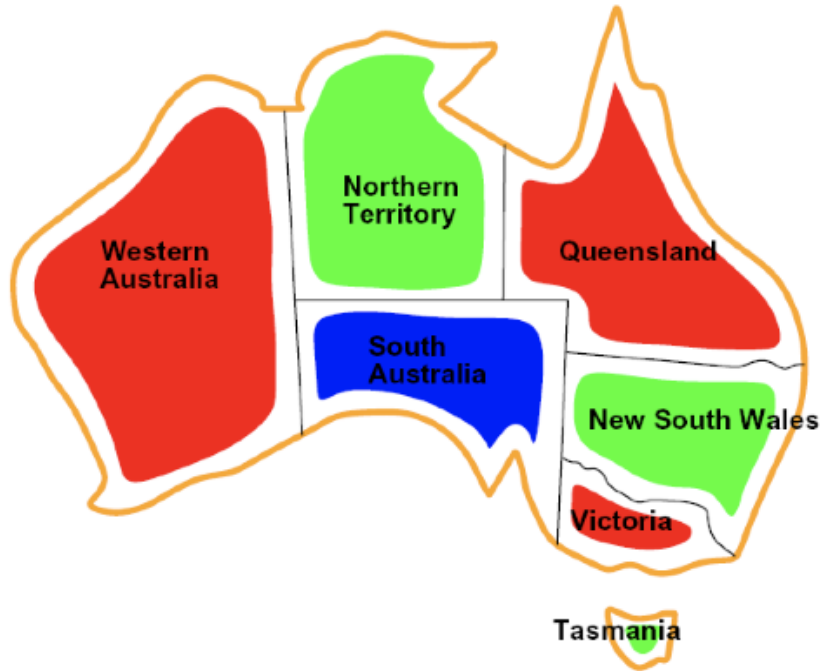
Example: N-Queens Problem



Tree Search of N-Queens Problem



Example: Map Coloring Problem



The Twist

- Goal states are complete states.
- All complete states exist at the lowest level of the search tree.
- Every path to a goal state is considered optimal (path optimality is irrelevant).
- Previous concepts (A^* , heuristics, etc.) no longer apply.
- Question: Why not just search exclusively among complete states?
- Result: Tree searching is no longer necessary.



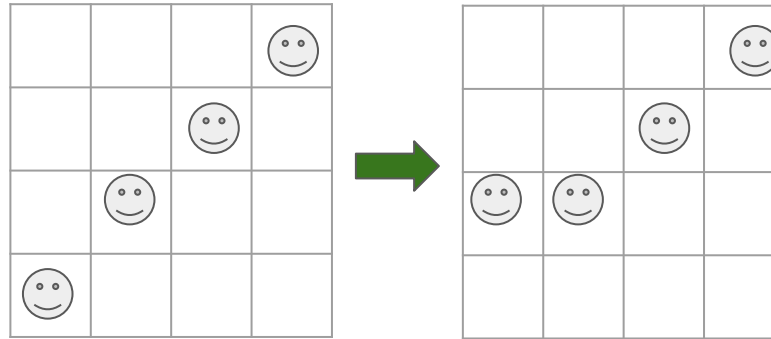
Local Search

- State Space: All complete configurations (complete states)
- Quiz: State Space size for N-Queens?
- State State: A random complete state
- The local search algorithm is just an iterative improvement: Keep a single "current" state, try to improve it by going to a neighbor state
- Local search improves a single option at each time (no frontier)
- Memory-less: only remember current state



Neighboring in Local Search

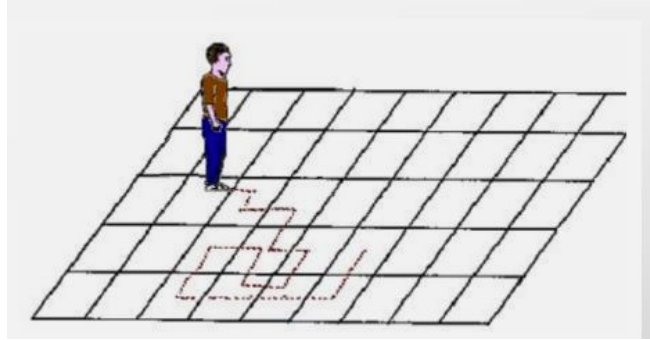
- Neighbors of a state: States that are different in only one variable



- Question: Which neighbor must be choose?



Random Walk

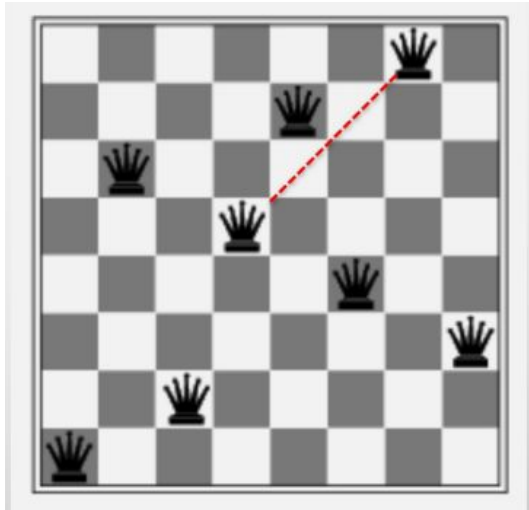


- Randomly pick a neighbor of the current state
- Random Walk algorithm is asymptotically complete (as the number of iterations increases, the probability that the algorithm has found the goal state approaches 1)



Fitness Function

- How good is each state?
- For N-Queens: # pairs of incompatible queens
- What about map coloring problem?



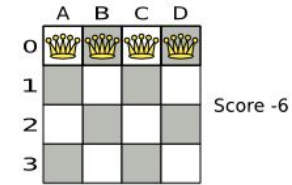
Hill-climbing

- Start from a random complete state
- Repeat: move to the best neighbor that is better than current state
- If no neighbors better than current state, quit
- Also called steepest-ascent hill climbing
- Complete?

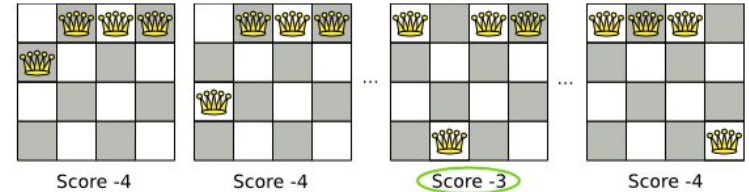


Hill-climbing on N-Queens

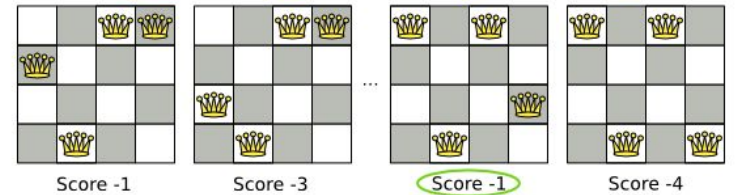
Step 0



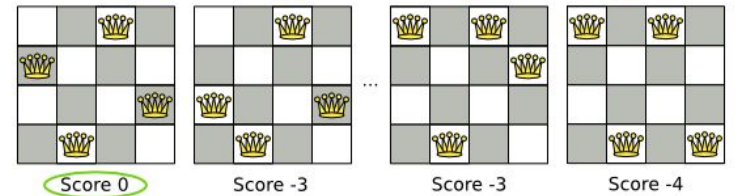
Step 1



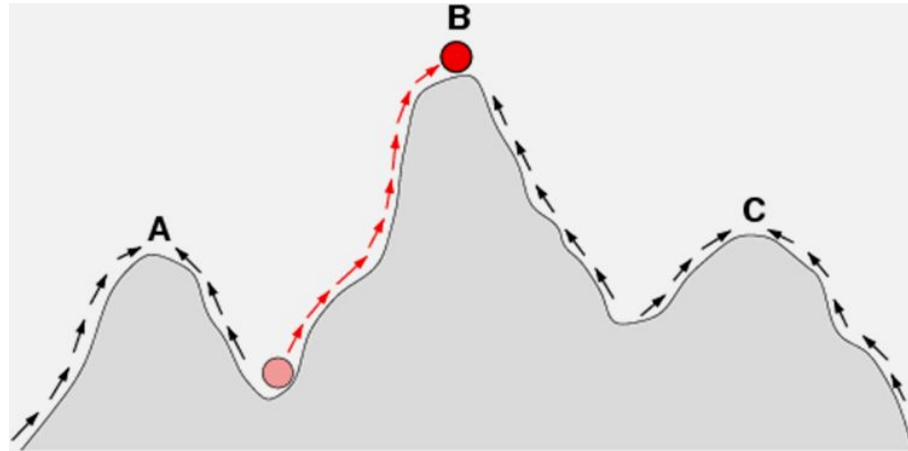
Step 2



Step 3



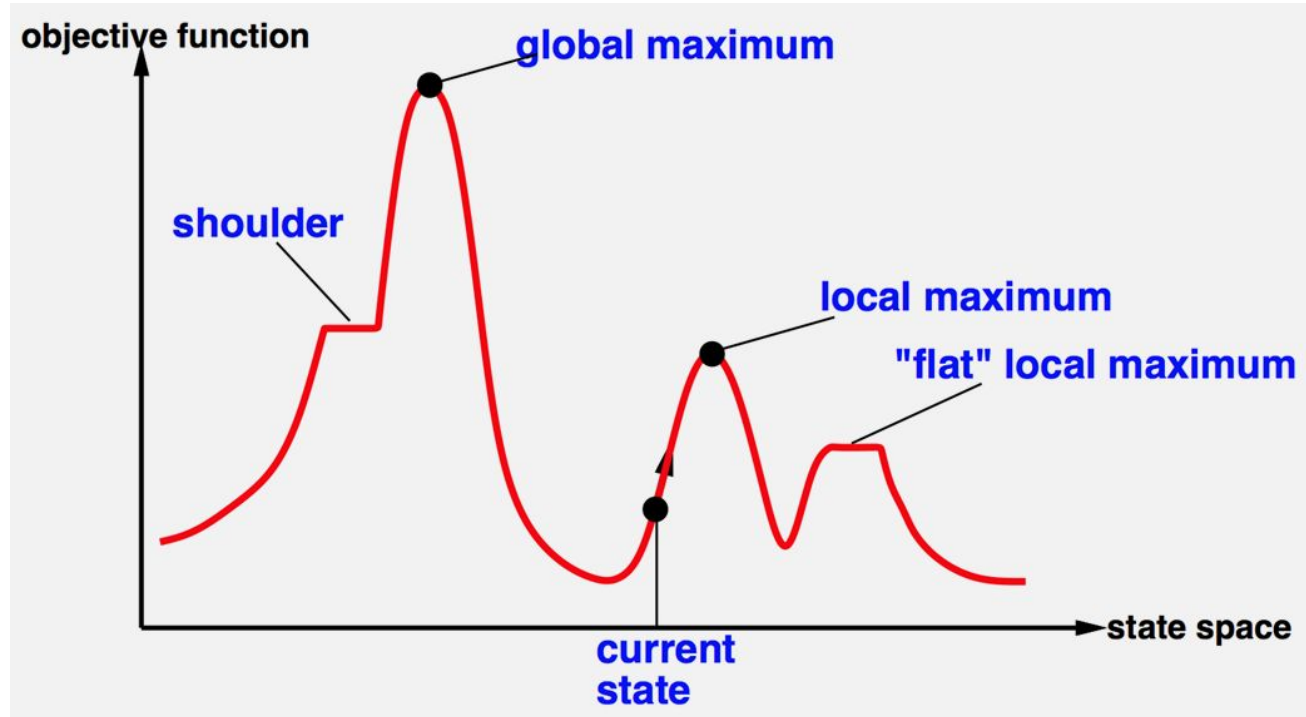
Hill-climbing Search is Greedy



Optimal when starting
in one of these states



Hill Climbing Hells



Hill-climbing Performance on n-queens

- Hill-climbing can solve large instances of n-queens ($n=106$) in a few seconds.
- 8 queens statistics:
 - State space of size ≈ 17 million
 - Starting from random state, steepest-ascent hill climbing solves 14% of problem instances
 - It takes 4 steps on average when it succeeds, 3 when it gets stuck



Escaping Shoulders: Sideways Move

- Sideways move: plateau may be a shoulder so keep going sideways moves when there is no uphill move
- Problem: infinite loop where flat local max
- Solution: upper bound on the number of consecutive sideways moves
- Result on 8-queens (Limit = 100 for consecutive sideways moves)
 - 94% success instead of 14% success
 - On average, 21 steps when succeeding and 64 steps when failing



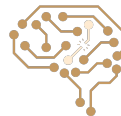
Hill Climbing Extensions: Tabu Search

- Prevent returning quickly to the same state
- Keep fixed length queue (“tabu list”)
- Add most recent state to queue; drop oldest
- Never move to a tabu state
- As the size of the tabu list grows, hill-climbing will asymptotically become “non redundant”
- In practice, a reasonable sized tabu list (say 100 or so) improves the performance of hill climbing in many problems



Hill Climbing: Stochastic Variations

- When the state-space landscape has local minima, any search that moves only in the greedy direction cannot be complete
- Random walk, on the other hand, is asymptotically complete
- Idea: Combine random walk & greedy hill-climbing
- At each step do one of the following:
 - Greedy : With prob. p move to the neighbor with largest value
 - Random walk: With prob. $1-p$ move to a random neighbor



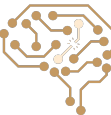
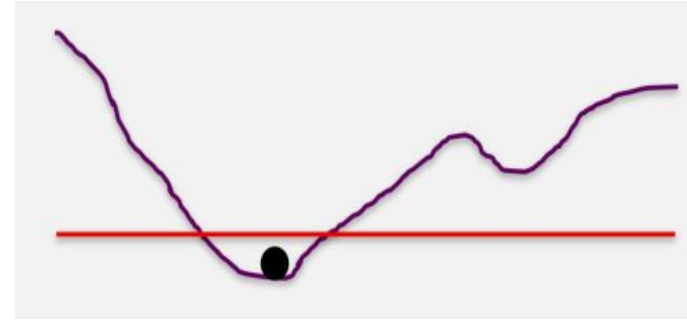
Hill-climbing with random restarts

- If at first you don't succeed, try, try again!
- Analysis
 - Say each search has probability p of success e.g., for 8-queens, $p = 0.14$ with no sideways moves
- Expected number of restarts?
- Expected number of steps taken?



Simulated Annealing (SA)

- Idea: escape local maxima by allowing some “bad” moves
- but gradually decrease their size and frequency
- Imagine letting a ball roll downhill on the function surface
- Now shake the surface, while the ball rolls
- Gradually reducing the amount of shaking



Simulated Annealing (SA) Algorithm

function Simulated-Annealing(*problem*, *schedule*) **returns**
solution state

current $\leftarrow$ Make-Node(Initial-State[*problem*])

for *t* $\leftarrow$ 1 **to** *infinity*

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then** **return** *current*

next $\leftarrow$ Random-Successor(*current*)

$\Delta E \leftarrow$ f-Value[*next*] - f-Value[*current*]

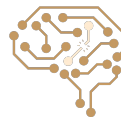
if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* with probability $e^{\Delta E/T}$

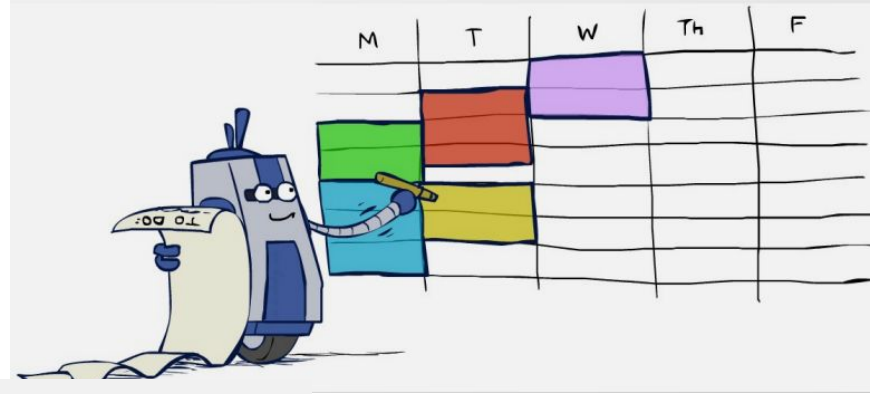
end

- ▶ Pick a random successor of the current state
- ▶ If it is better than the current state go to it
- ▶ Otherwise, accept the transition with a probability

$T(t) = \text{schedule}[t]$ is a decreasing series



Next Session: Local Search & CSP



TWO
+ TWO

FOUR

